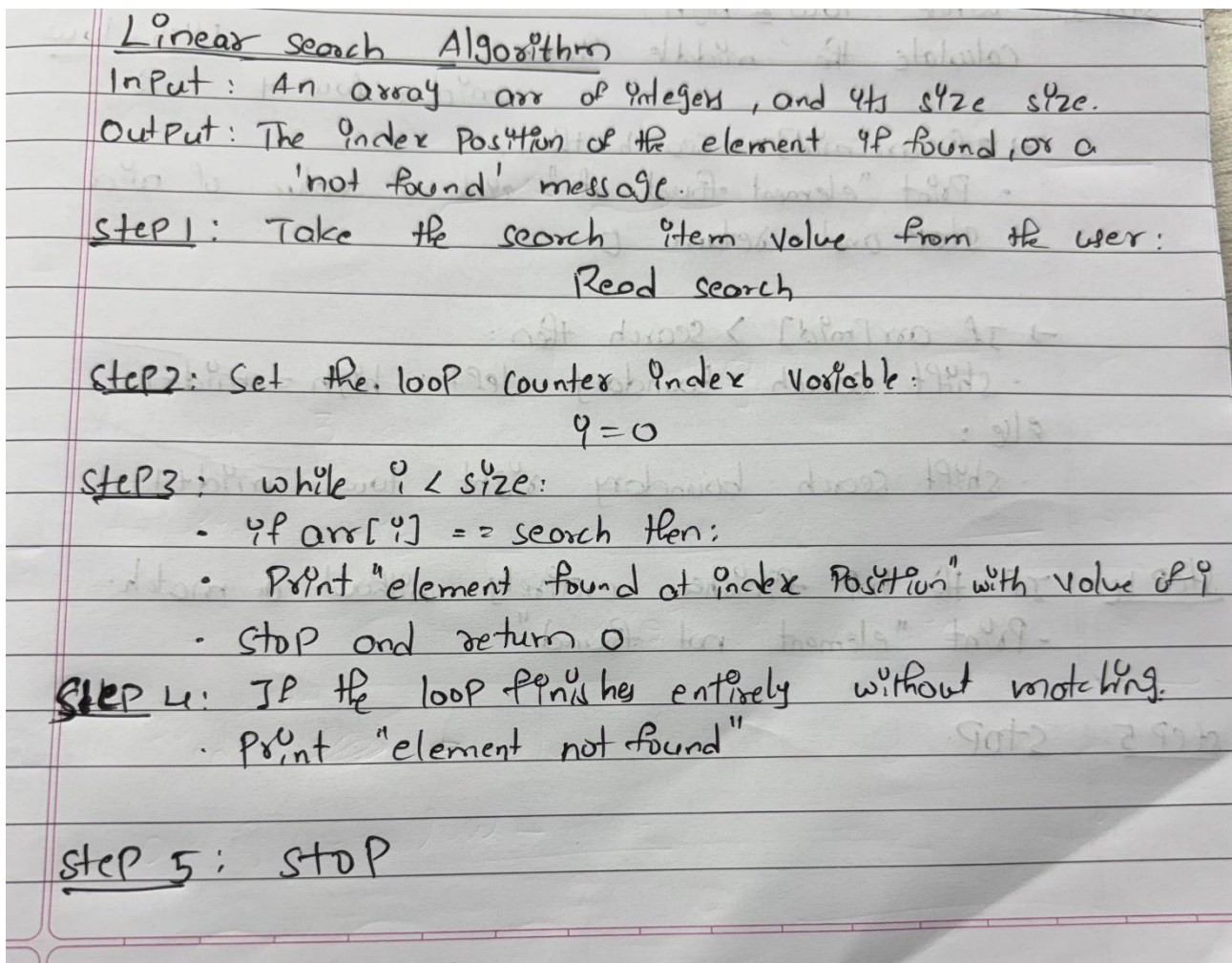


Practical - 2

Aim : To implementation and Time analysis of Linear and Binary search Algorithms.

1) Linear Search

Algorithm :



Linear Search Algorithm

Input: An array arr of integers, and its size size.

Output: The index position of the element if found, or a 'not found' message.

Step 1: Take the search item value from the user:
Read search

Step 2: Set the loop counter index variable:
 $i = 0$

Step 3: while $i < \text{size}$:
- if $\text{arr}[i] == \text{search}$ then:
- Print "element found at index position" with value of i
- stop and return 0

Step 4: If the loop finishes entirely without matching.
- Print "element not found"

Step 5: stop

Program / Code :

```
#include<iostream>
using namespace std;

class LinearSearch{
public:
    int size, arr[100], i;

    void arrayInput(){
        cout << "enter the size of array" << endl;
        cin >> size;

        cout << "enter array elements" << endl;
        for(i = 0; i < size; i++){
            cin >> arr[i];
        }
    }

    void PrintArray(){
        for(i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
    }

    int LinearSearchLogic(){
        int search;

        cout << "Enter element to search: ";
        cin >> search;

        for(i = 0; i < size; i++){
            if(arr[i] == search){
                printf("element found at %d", i);
                return 0;
            }
        }
    }
}
```

```
}  
    }  
  
    printf("element not found");  
    return 0;  
}  
};  
  
int main(){  
    LinearSearch Ls;  
  
    Ls.arrayInput();  
  
    cout << "\nArray Elements" << endl;  
    Ls.PrintArray();  
  
    cout << endl;  
    Ls.LinearSearchLogic();  
  
    return 0;  
}
```

Output :

```
C:\Users\NIKSHITH>cd "C:\Users\NIKSHITH\DAA"

C:\Users\NIKSHITH\DAA>linear_search.exe
enter the size of array
6
enter array elements
9
5
3
8
1
4

Array Elements
9 5 3 8 1 4
Enter element to search: 4
element found at 5
C:\Users\NIKSHITH\DAA>
```

Time Complexity :

- **Best Case** : $O(1)$
- **Average Case** : $O(n)$
- **Worst Case** : $O(n)$

Linear search Time complexity:

Time to search in array size n :

$$T(n) = T(n-1) + O(1)$$

$$T(n) = T(n-1) + 1 = T(n-1) + 1 + 1 + \dots + T(1) + (n-1)$$

Best Case : $T(1) = O(1)$

$T(n) = O(n)$

* Best Case : $O(1)$

* Worst Case : $O(n)$

* Average Case : $O(n)$

2) Binary Search

Algorithm :

Binary Search Algorithm

Input : A sorted array arr of integers, and size 'size'
Output : The index position of the element if found, or a 'not found' message.

Step 1: Take the search item value from the user:
Read search

Step 2: set search boundary range pointers:
low = 0
high = size - 1

Step 3: while low $\leq$ high:
calculate the middle index: $mid = low + \frac{high - low}{2}$
→ If arr[mid] == search then:
• Print "element found at" side of value of mid.
• stop and return 0.
→ If arr[mid] > search then:
• shift search boundary left: high = mid - 1
Else:
• shift search boundary right: low = mid + 1

Step 4: If the loop finishes entirely without a match:
• Print "element not found"

Step 5: STOP.

Program / Code :

```
#include<iostream>
using namespace std;

class BinarySearch{
public:
    int size, arr[100], i;

    void arrayInput(){
        cout << "enter the size of array" << endl;
        cin >> size;

        cout << "enter array elements (in sorted order)" << endl;
        for(i = 0; i < size; i++){
            cin >> arr[i];
        }
    }

    void PrintArray(){
        for(i = 0; i < size; i++){
            cout << arr[i] << " ";
        }
    }

    int BinarySearchLogic(){
        int search;
        int low = 0;
        int high = size - 1;
        int mid;

        cout << "Enter element to search: ";
        cin >> search;
        while(low <= high){
            mid = low + (high - low) / 2;
```

```
if(arr[mid] == search){
    printf("element found at %d", mid);
    return 0;
}
if(arr[mid] > search){
    high = mid - 1;
}

else{
    low = mid + 1;
}
}
printf("element not found");
return 0;
}
};
```

```
int main(){
    BinarySearch Bs;
    Bs.arrayInput();
    cout << "\nArray Elements" << endl;
    Bs.PrintArray();
    cout << endl;
    Bs.BinarySearchLogic();
    return 0;
}
```

Output :

```
C:\Users\NIKSHITH\DAA>g++ "Binary Search.cpp" -o binary_search.exe

C:\Users\NIKSHITH\DAA>binary_search.exe
enter the size of array
5
enter array elements (in sorted order)
1
3
4
7
9

Array Elements
1 3 4 7 9
Enter element to search: 7
element found at 3
```

Time Complexity :

- **Best Case** : $O(1)$
- **Average Case** : $O(\log n)$
- **Worst Case** : $O(\log n)$

Time Complexity (Binary Search)

Let $T(n)$ = time to search in array of size n , for checking middle element (constant $O(1)$ search half the array)
 $\rightarrow n/2$.

$$T(n) = T\left(\frac{n}{2}\right) + 1 = T\left(\frac{n}{2}\right) + 1 + 1 \dots - T(1) + \log_2 n$$

$$\text{Best Case : } T(1) = O(1)$$

$$T(n) = O(\log n)$$

$$\text{Best Case} = O(1)$$

$$\text{Average Case} = O(\log n)$$

$$\text{Worst Case} = O(\log n)$$